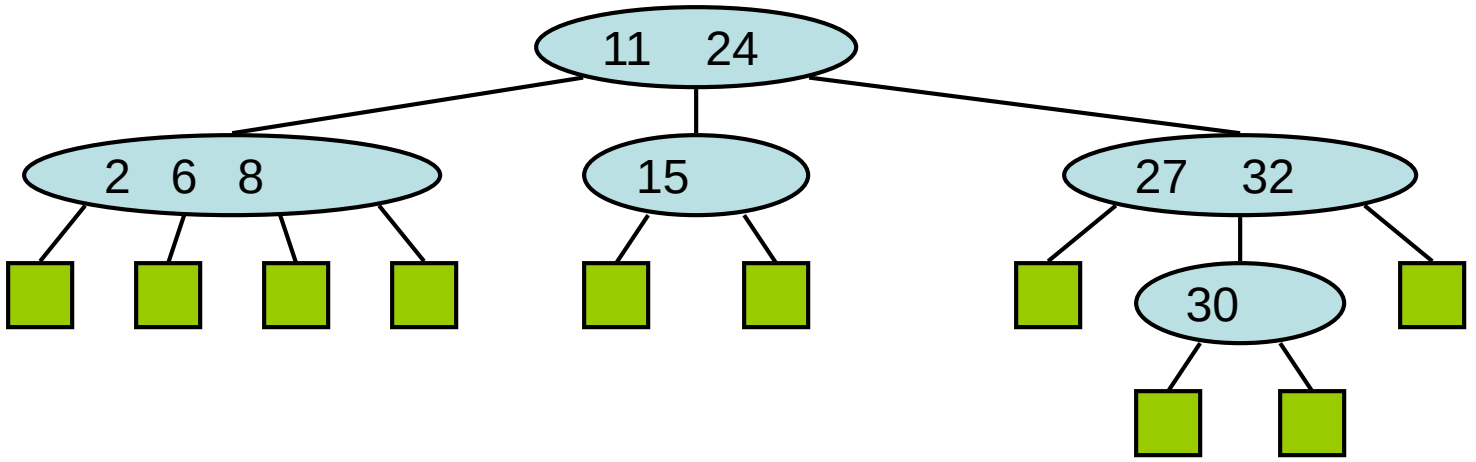
Multi-way Search Tree

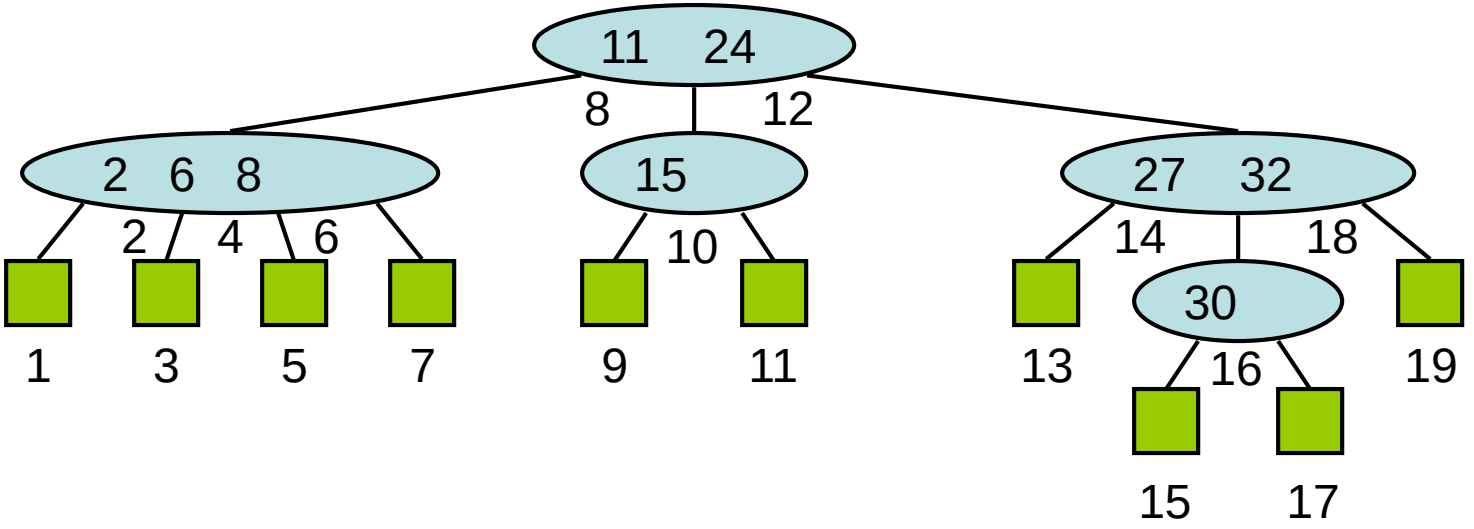
A multi-way search tree is an ordered tree such that

- Each internal node has at least two children and stores d -1 key-object (k_i, o_i), where d is the number of children
- For a node with children v_1 v_2 … v_d storing keys k_1 k_2 … k_{d-1}
  - keys in the subtree of v_1 are less than k_1
  - keys in the subtree of v_i are between k_{i-1} and k_i (i = 2, …, d - 1)
  - keys in the subtree of v_d are greater than k_{d-1}
- The leaves (external nodes) store no objects and serve as placeholders



Multi-way Inorder Traversal

- We can extend the notion of inorder traversal from binary trees to multi-way search trees.
- An inorder traversal of a multi-way search tree visits the keys in increasing order.

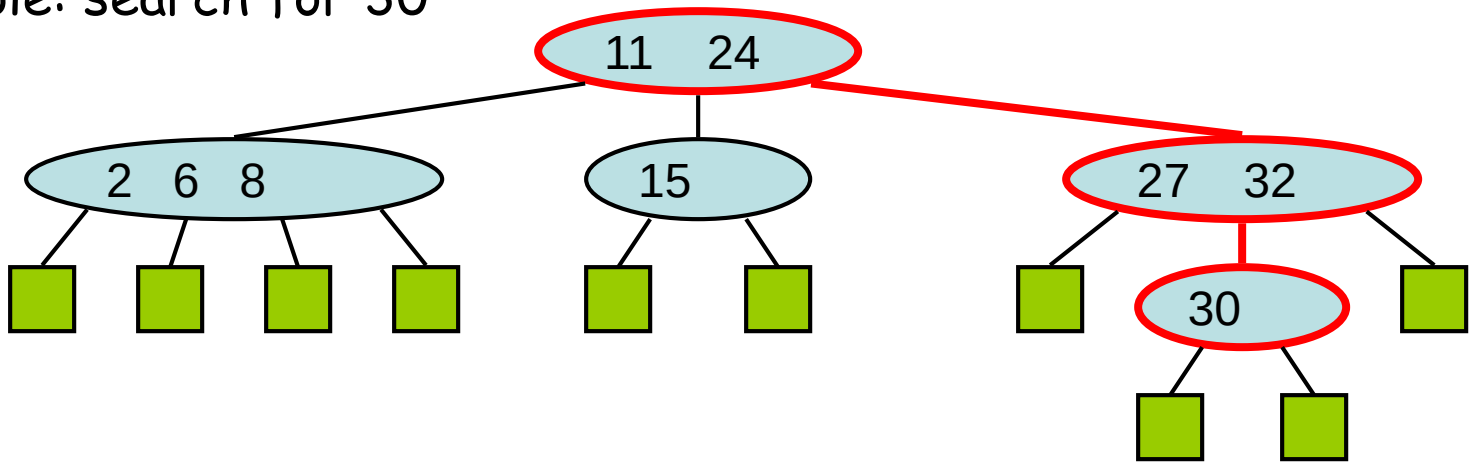


Multi-way Searching

- Similar to search in a binary search tree
- Each internal node with children $v_1\ v_2\ \ldots\ v_d$ and keys $k_1\ k_2\ \ldots\ k_{d-1}$
  - If $k = k_i$ ($i = 1, \ldots, d - 1$): the search terminates successfully
  - If $k < k_1$: continue the search in child v_1
  - If $k_{i-1} < k < k_i$ ($i = 2, \ldots, d - 1$): continue the search in child v_i
  - If $k > k_{d-1}$: continue the search in child v_d

- Reaching an external node terminates the search unsuccessfully

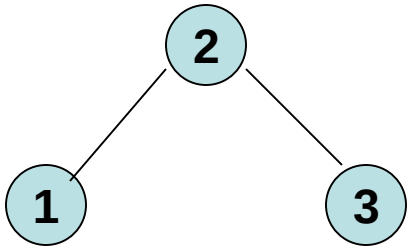
- Example: search for 30



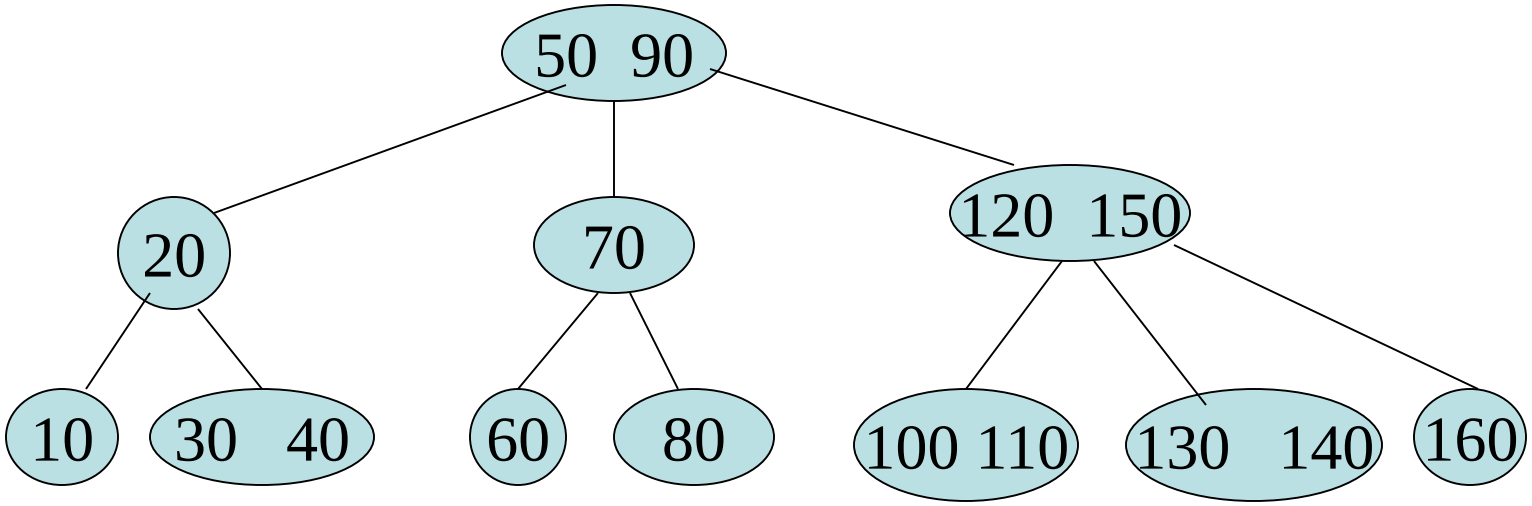
(2,3) Tree

It is a tree in which

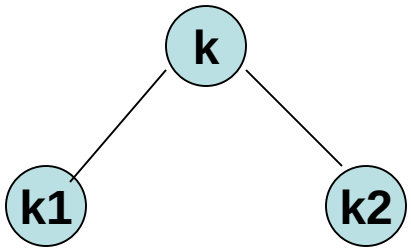
- Every non-leaf node has 2 or 3 children.
- Every path from root to leaf is of same length (external nodes are in same depth).
- All data are kept in sorted order.



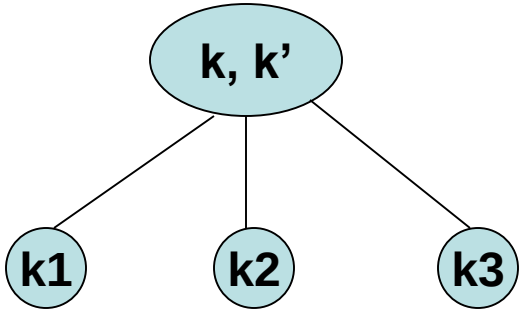
Example



- 2-Nodes have one data item and two children.

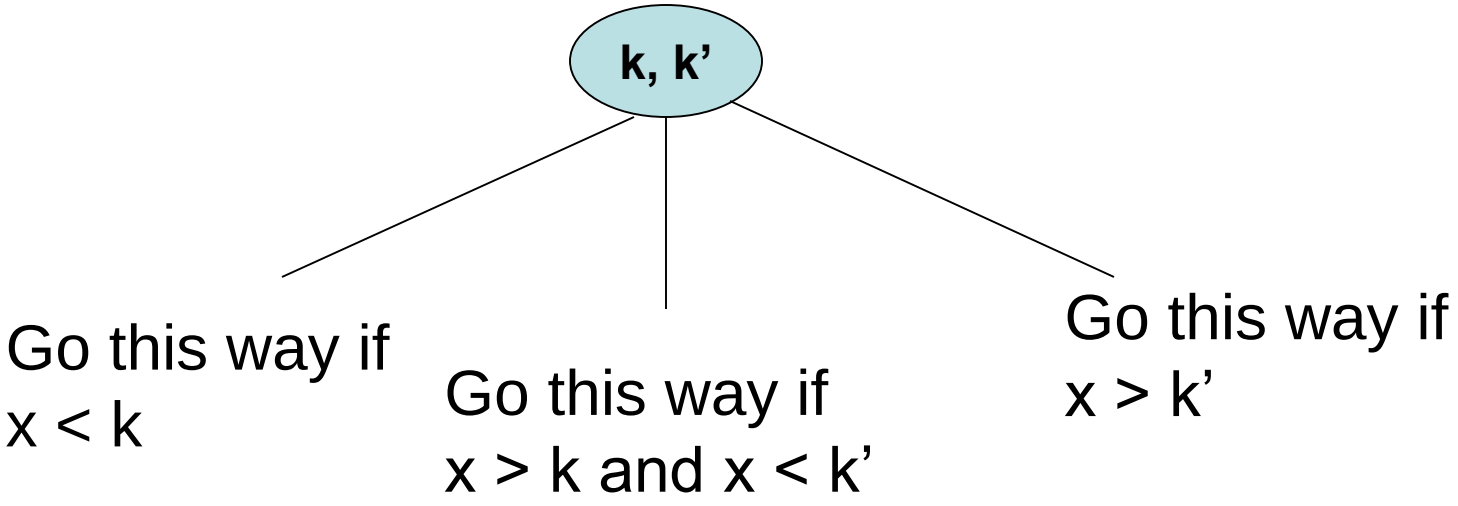


- 3-Nodes have two data items and three children.



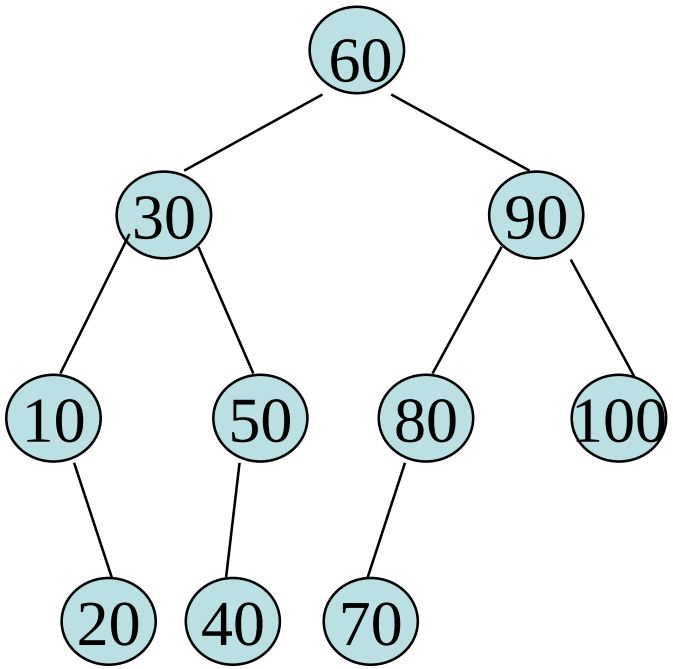
Searching element in (2,3) tree

- Searching is similar to BST.

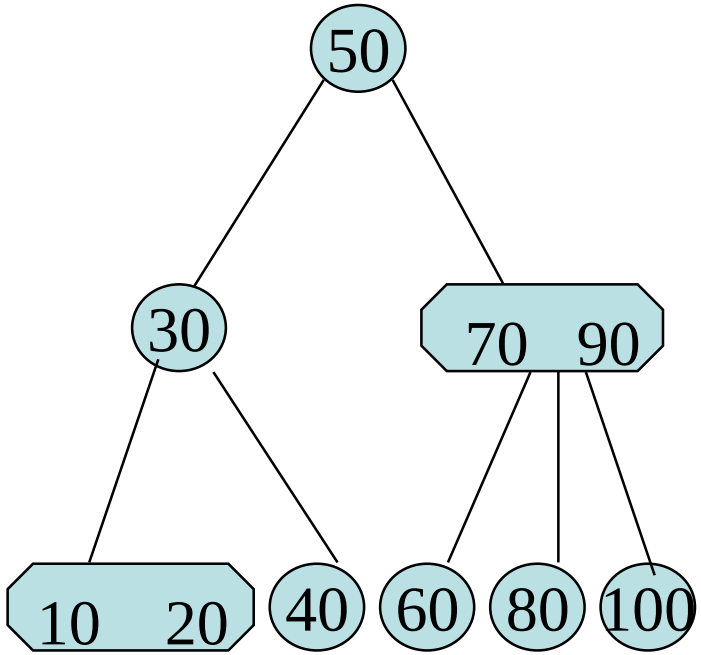


(2,3) vs. binary tree

Consider searching for item 20 in the following two trees



A balanced binary search tree

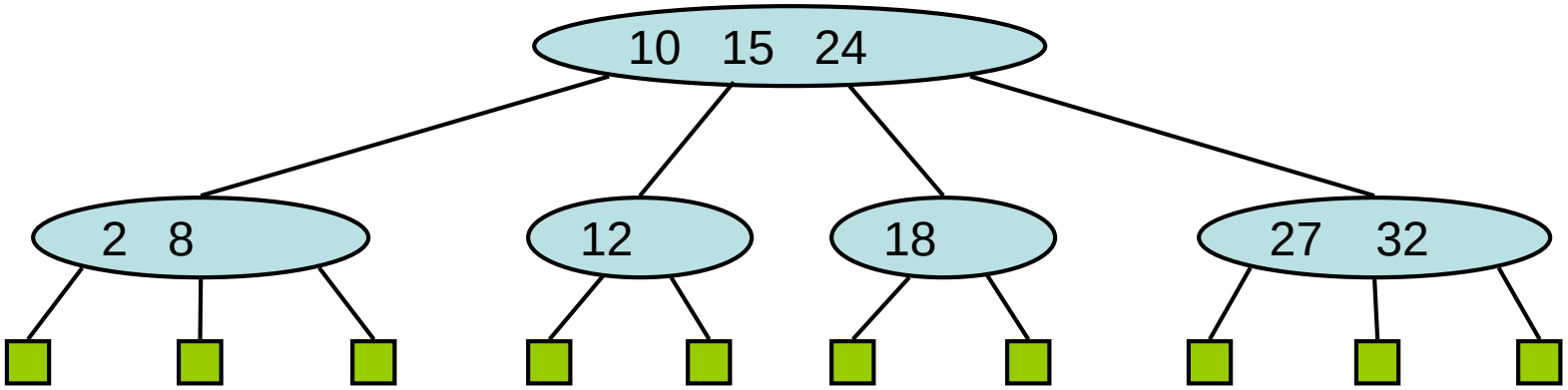


A (2,3) tree

(2,4) Tree

- A (2,4) tree (also called 2-4 tree or 2-3-4 tree) is a multi-way search with the following properties
  - Node-Size Property: every internal node has at most four children
  - Depth Property: all the external nodes have the same depth

- Depending on the number of children, an internal node of a (2,4) tree is called a 2-node, 3-node or 4-node



Height of a (2,4) Tree

Theorem: A (2,4) tree storing n items has height $O(\log n)$

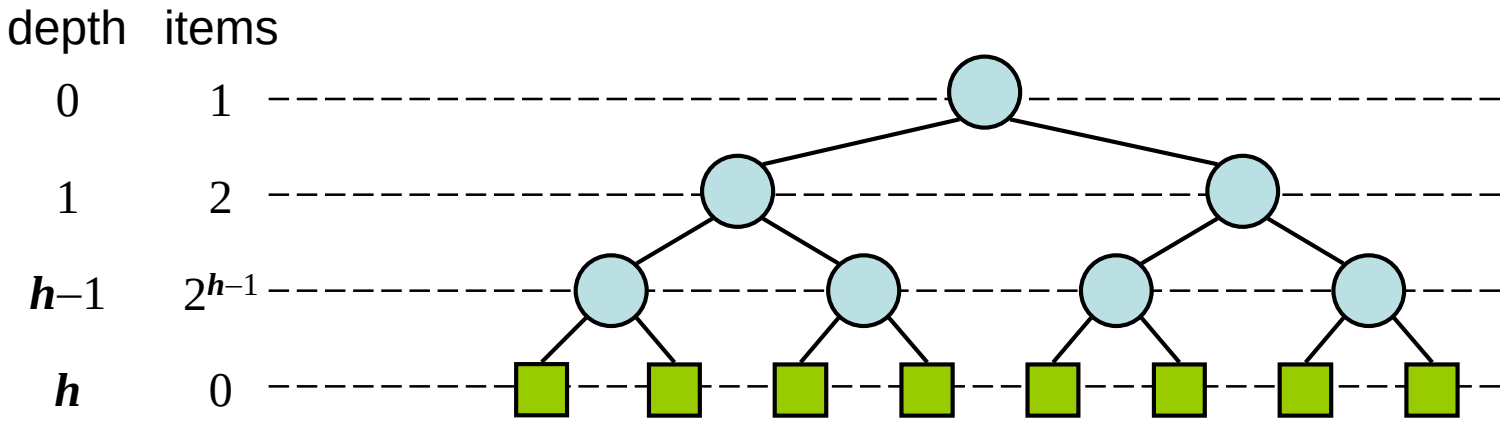
Proof:

- Let h be the height of a (2,4) tree with n items
- Since there are at least 2^i items at depth $i = 0, \ldots, h - 1$ and no items at depth h, we have

$$n \geq 1 + 2 + 4 + \ldots + 2^{h-1} = 2^h - 1$$

- Thus, $h \leq \log (n + 1)$

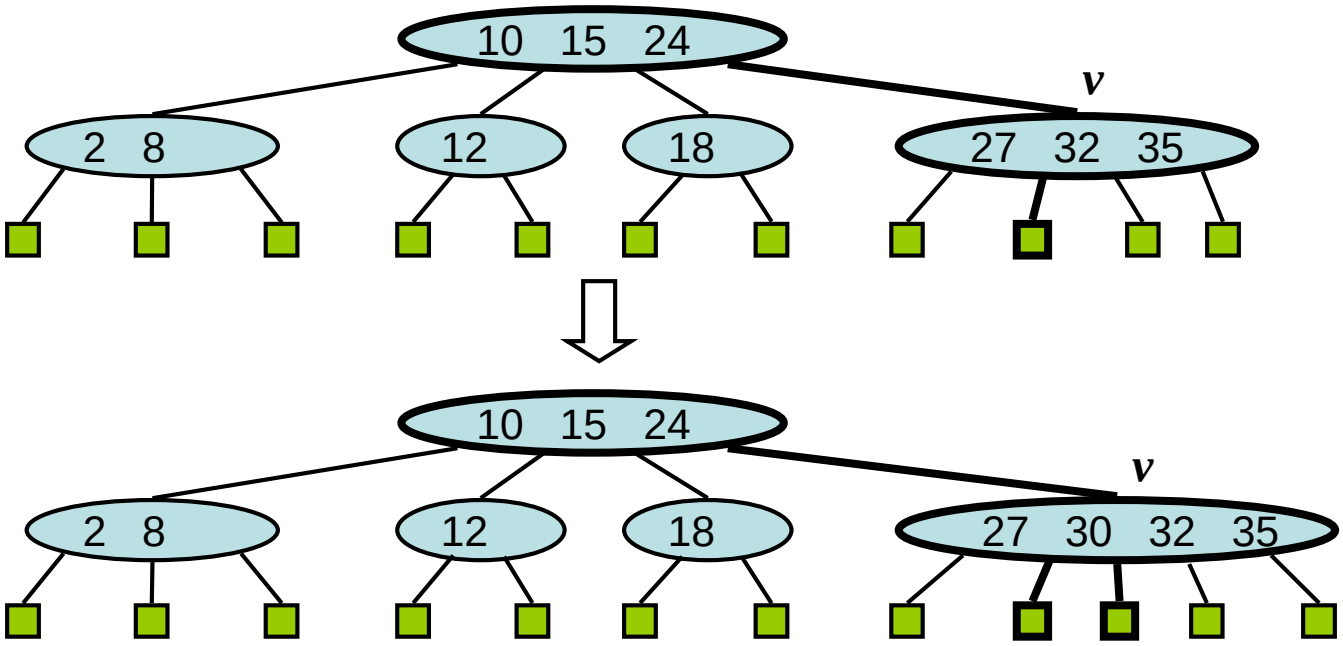
- Searching in a (2,4) tree with n items takes $O(\log n)$ time



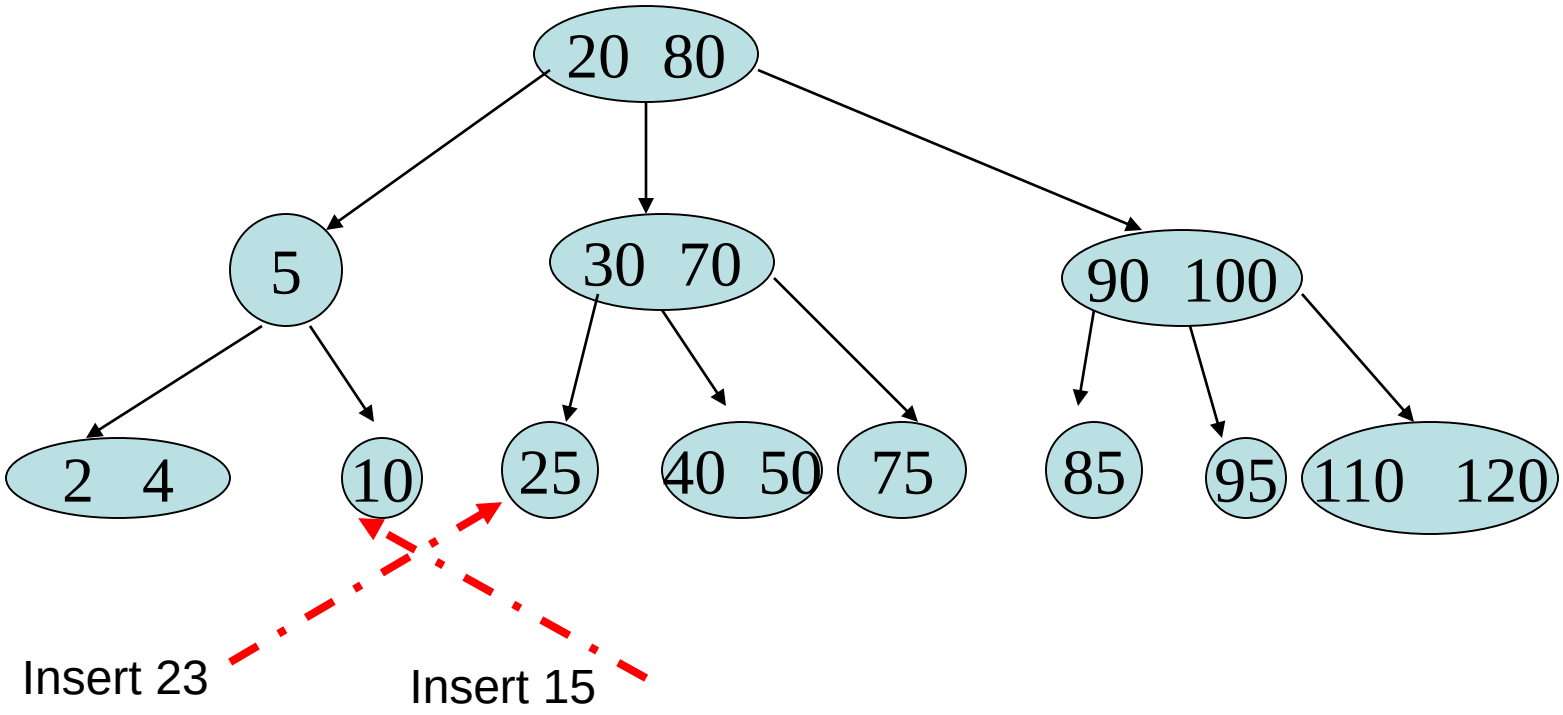
Insertion

- Insert a new item (***k***, ***o***) at the parent ***v*** of the leaf reached by searching for ***k***
  - Need to preserve the depth property
  - May cause an overflow (i.e., node ***v*** may become a 5-node)

- Example: inserting key 30 causes an overflow

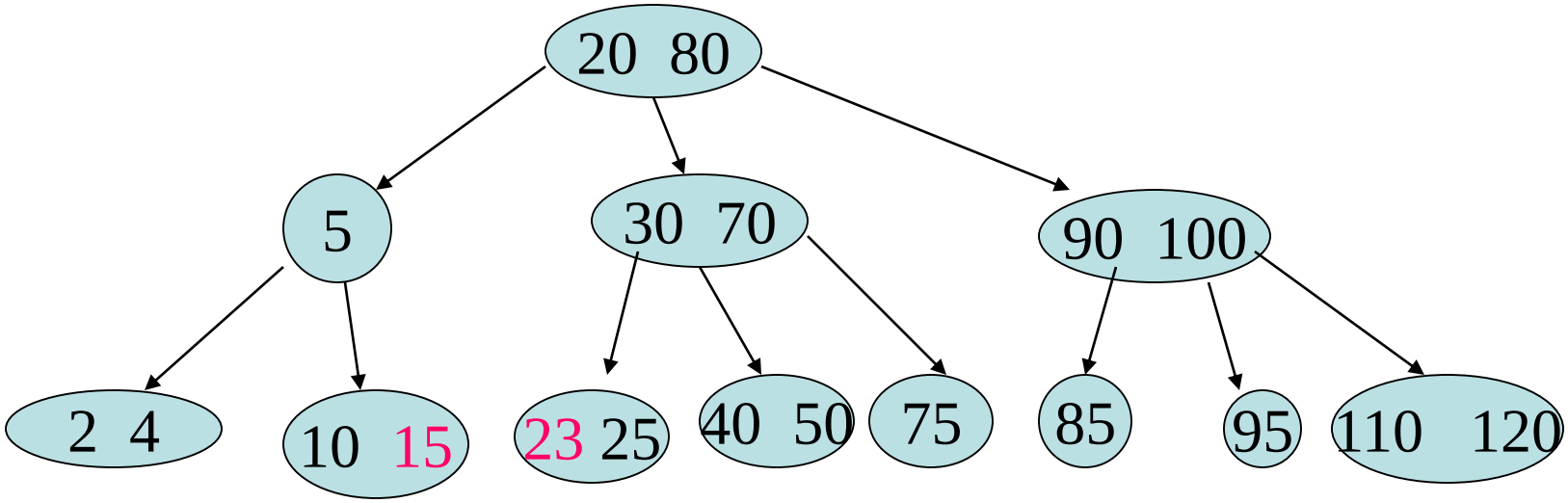


Insertion example



Insertion example

After inserting 23 and 15

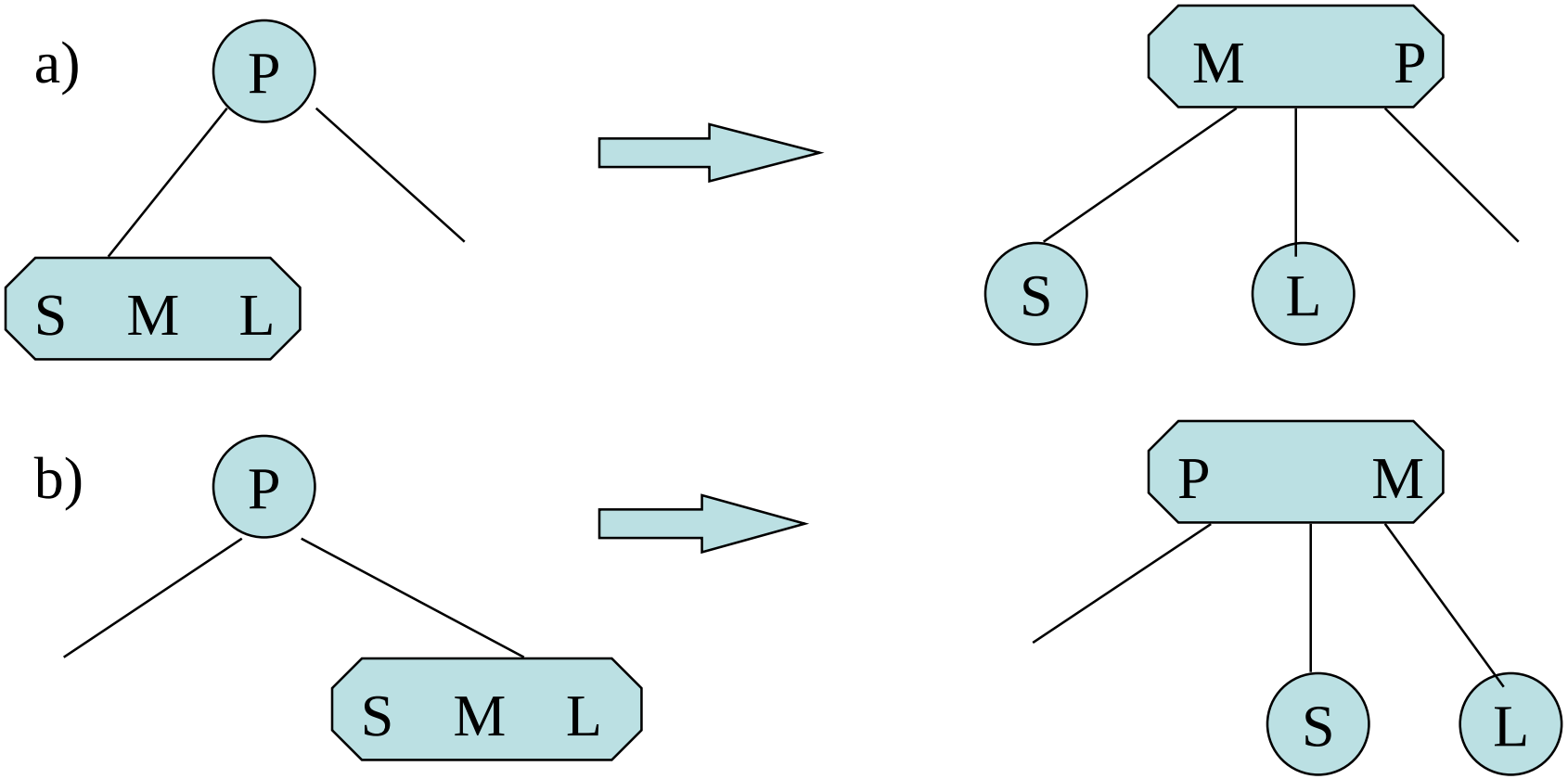


Note: Insertion example

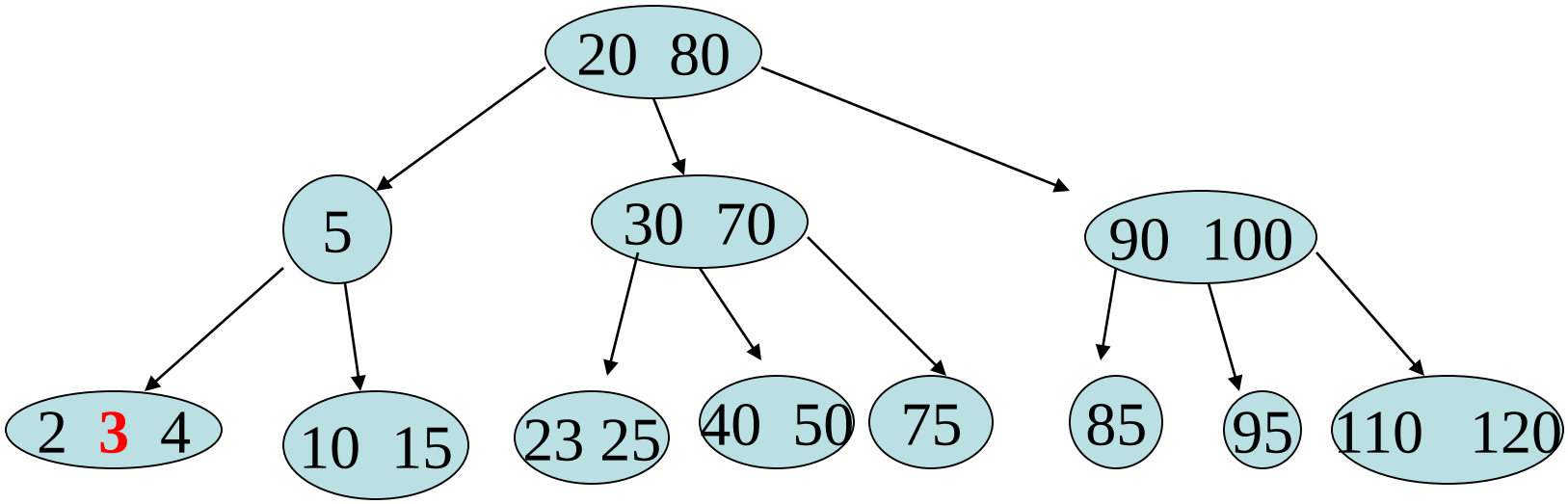
- If the node is a 3-node, you cannot simply add another value.
- Split the leaf: replace the 3-node by two 2-node and push the middle value up to the parent node.
- Repeat recursively until you upgrade a 2-node or create a new root.

Insertion example

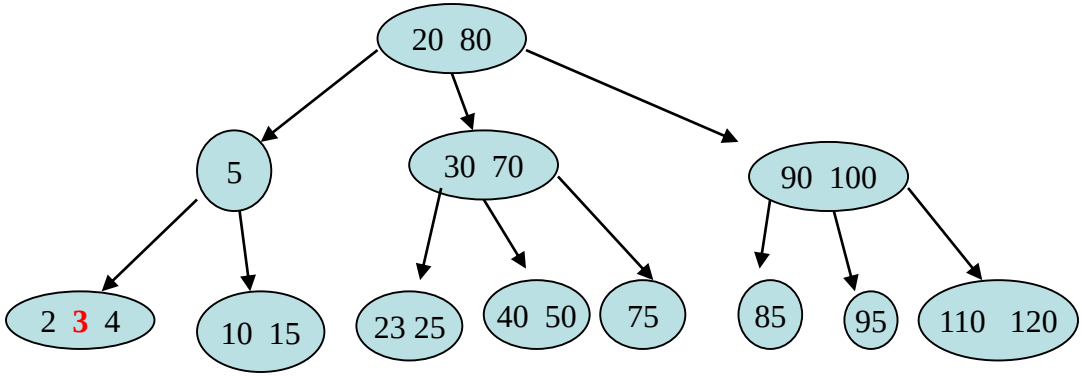
Splitting a node

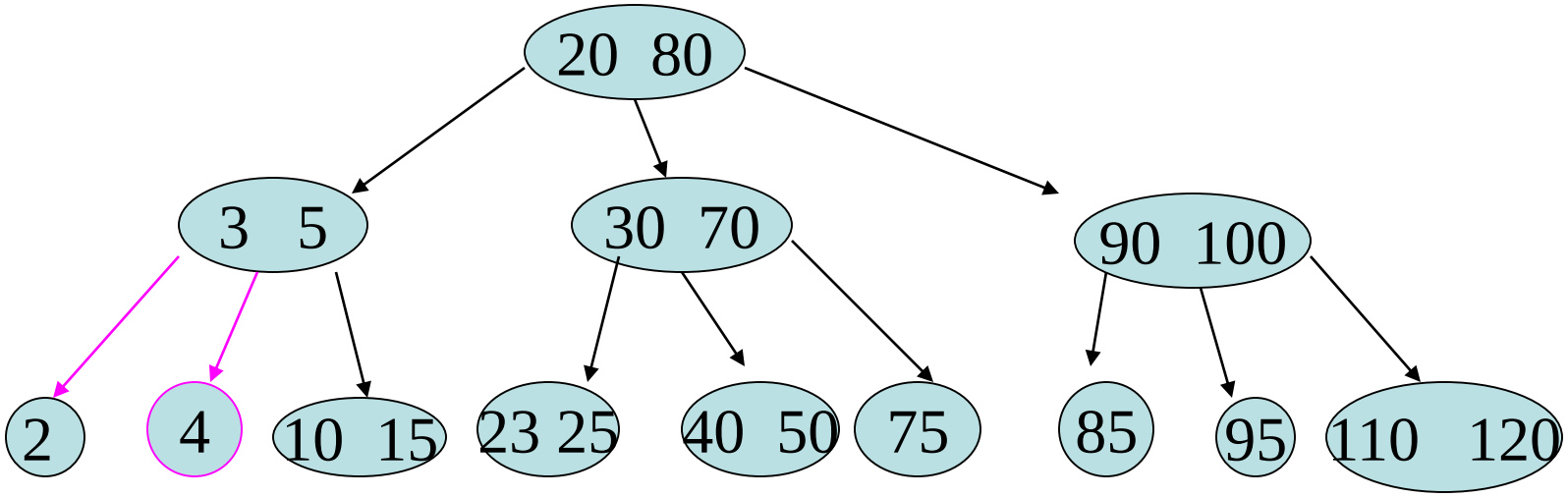


Splitting a node



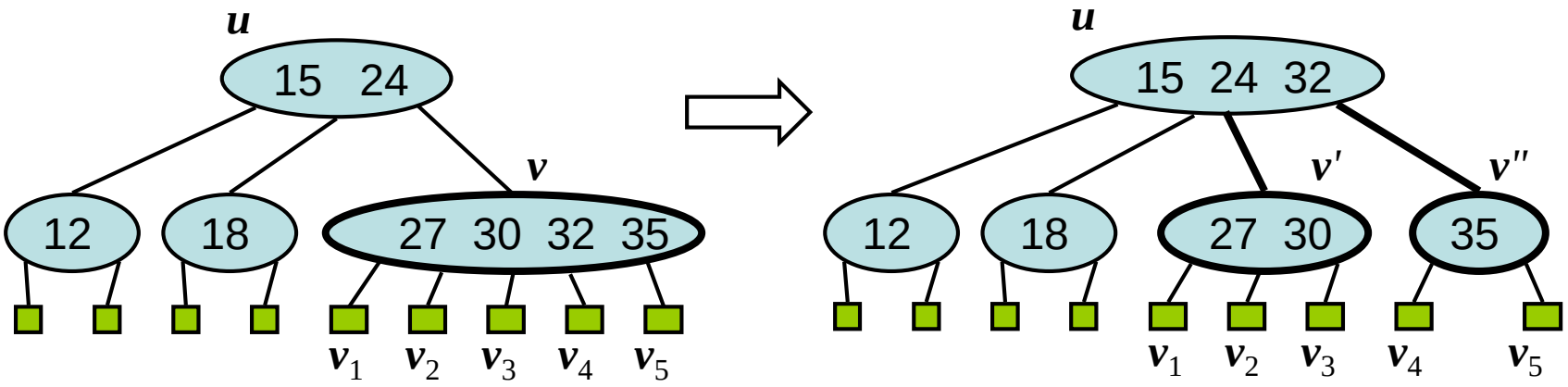
After splitting





Overflow and Split

- We handle an overflow at a 5-node v with a split operation:
  - let v_1 … v_5 be the children of v and k_1 … k_4 be the keys of v
  - node v is replaced by nodes v' and v''
    - v' is a 3-node with keys k_1 k_2 and children v_1 v_2 v_3
    - v'' is a 2-node with key k_4 and children v_4 v_5
  - key k_3 is inserted into the parent u of v
- The overflow may propagate to the parent node of u (a new root may be created, if needed)



Analysis of Insertion Operation

Algorithm ***insertItem***(***k***, ***o***)

1. We search for key ***k*** to locate the insertion node ***v***

2. We add the new item (***k***, ***o***) at node ***v***

3. **while** ***overflow***(***v***)
   if ***isRoot***(***v***)
   create a new empty root above ***v***
   v ← ***split***(***v***)

- Let ***T*** be a (2,4) tree with ***n*** items
  - Tree ***T*** has ***O***(log ***n***) height
  - Step 1 takes ***O***(log ***n***) time
  - Step 2 takes ***O***(1) time
  - Step 3 takes ***O***(log ***n***) time because each split takes ***O***(1) time and we may perform ***O***(log ***n***) splits
- Thus, an insertion in a (2,4) tree takes ***O***(log ***n***) time

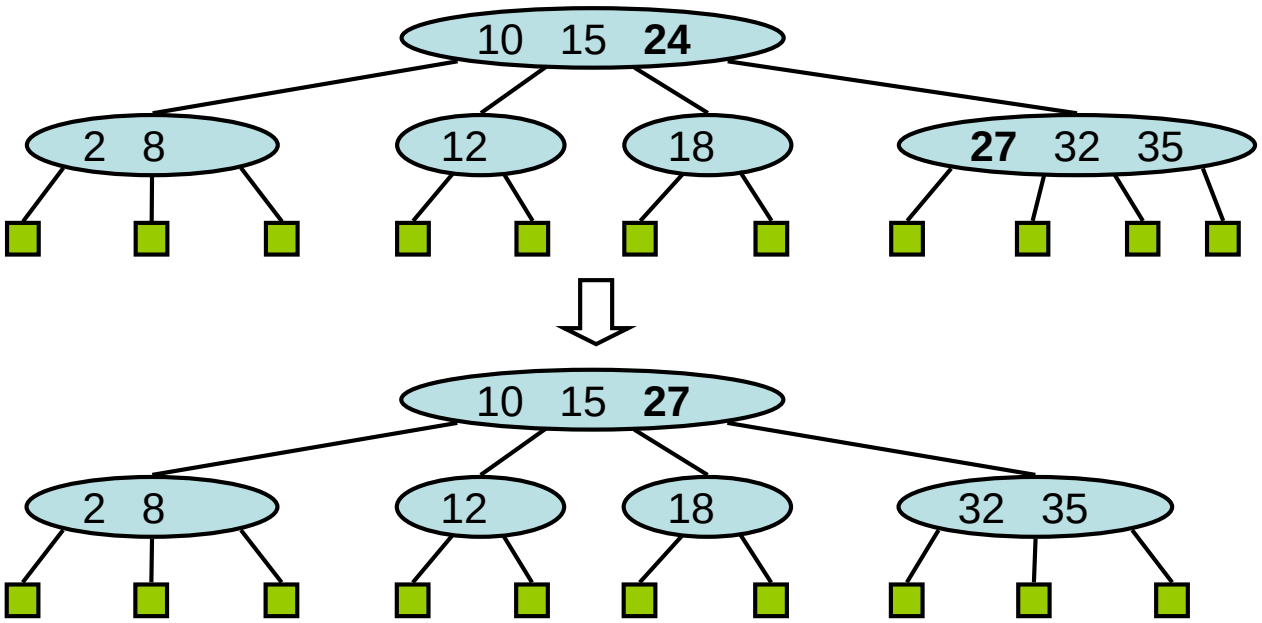
Deleting from a (2,3) tree

- The deletion algorithm always deletes an item from a leaf.
- If the item to be deleted **is not in a leaf**, then exchange the item with **its inorder successor.**
- Then, delete item from the leaf.

Deletion

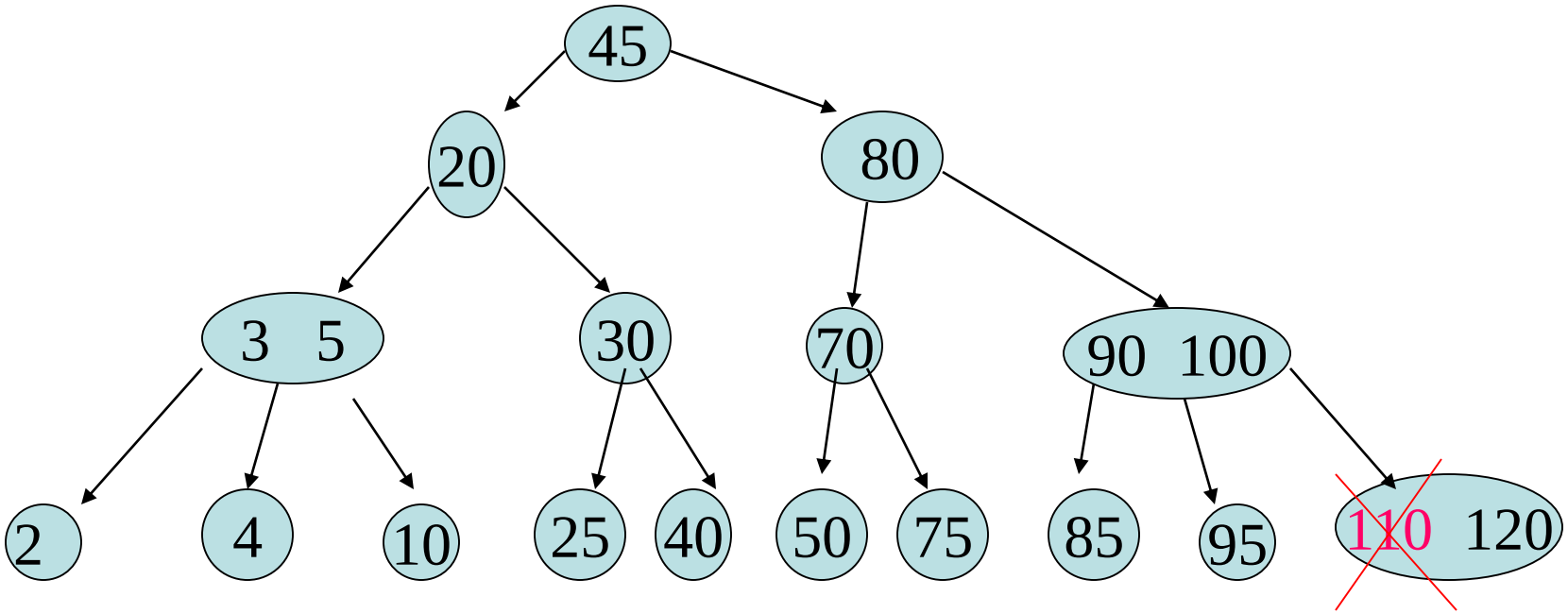
If the item is at the node with leaf children, delete the item;
Else, we replace the item with its inorder successor and delete the item.

Example: to delete key 24, we replace it with 27 (inorder successor)



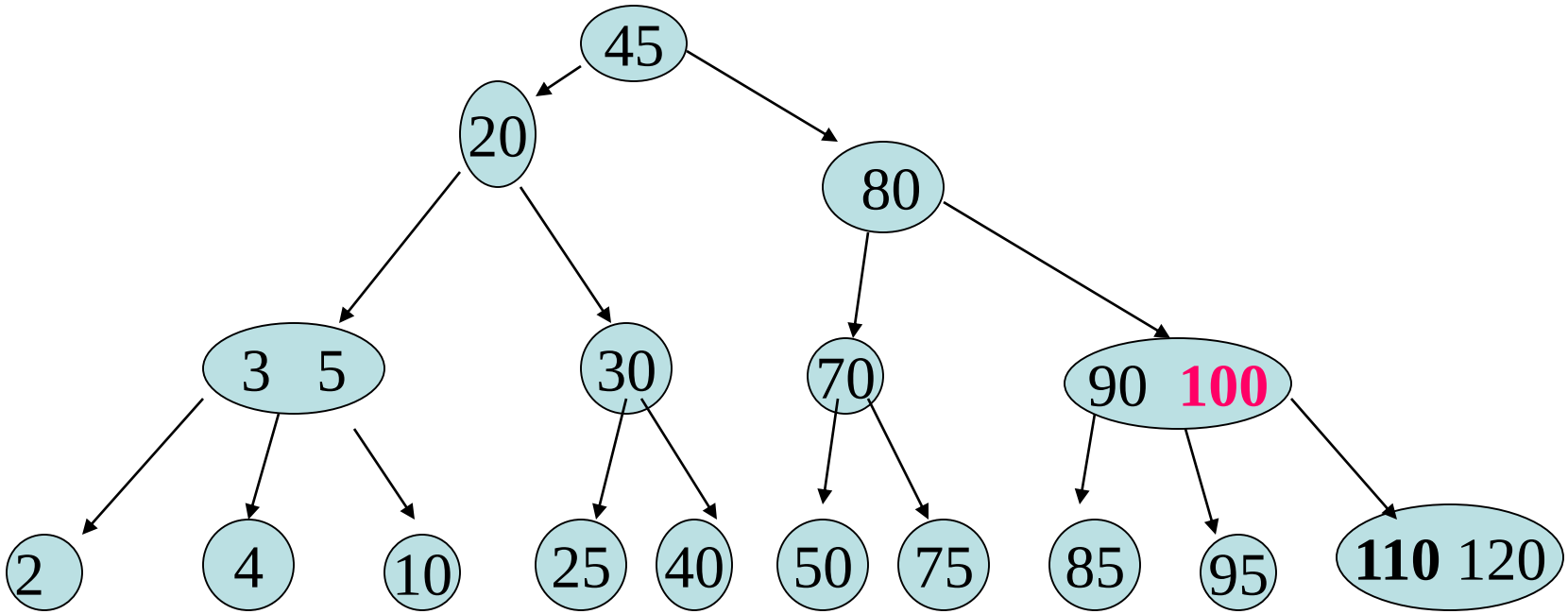
Deletion when the item is at a node with leaf children

Example: Remove(110)



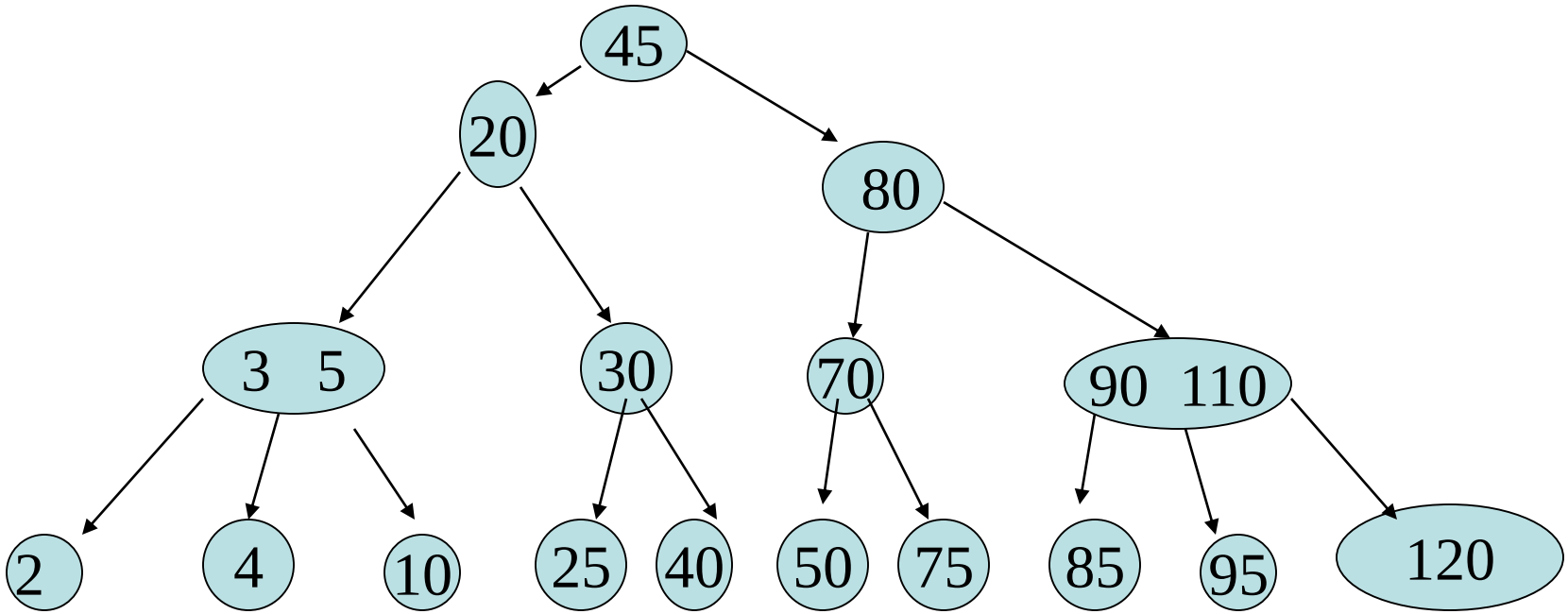
Remove(100)

- Step 1: Exchange 100 with inorder successor



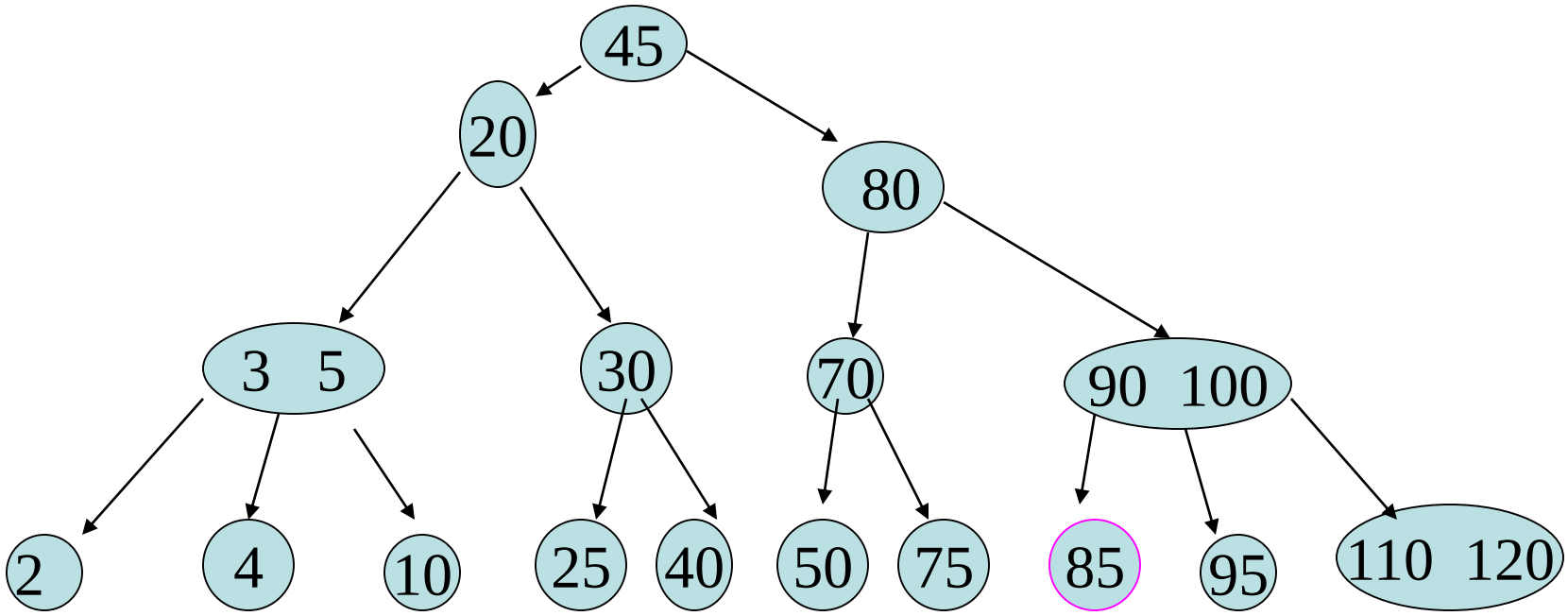
Remove(100)

- Step 2: Remove(100)

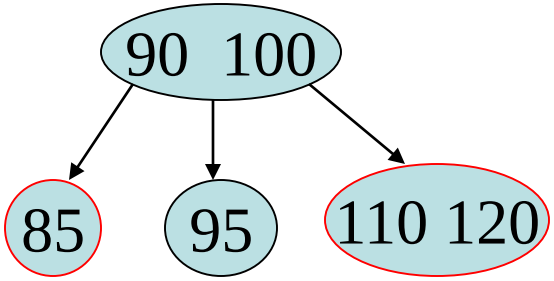


More on Deletion…

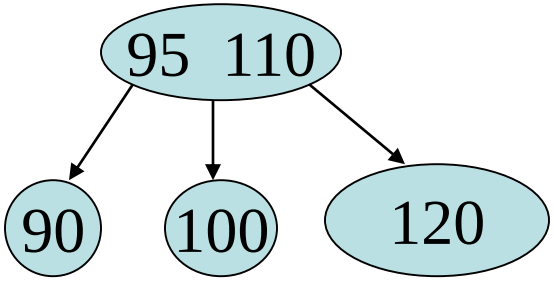
- Suppose the item to be deleted is in a node that contains no other item.
- If the node has a sibling that contains 2 items (3-node), we can rearrange using the parent.
- Example: Remove(85)



Remove(85)



Delete value from leaf

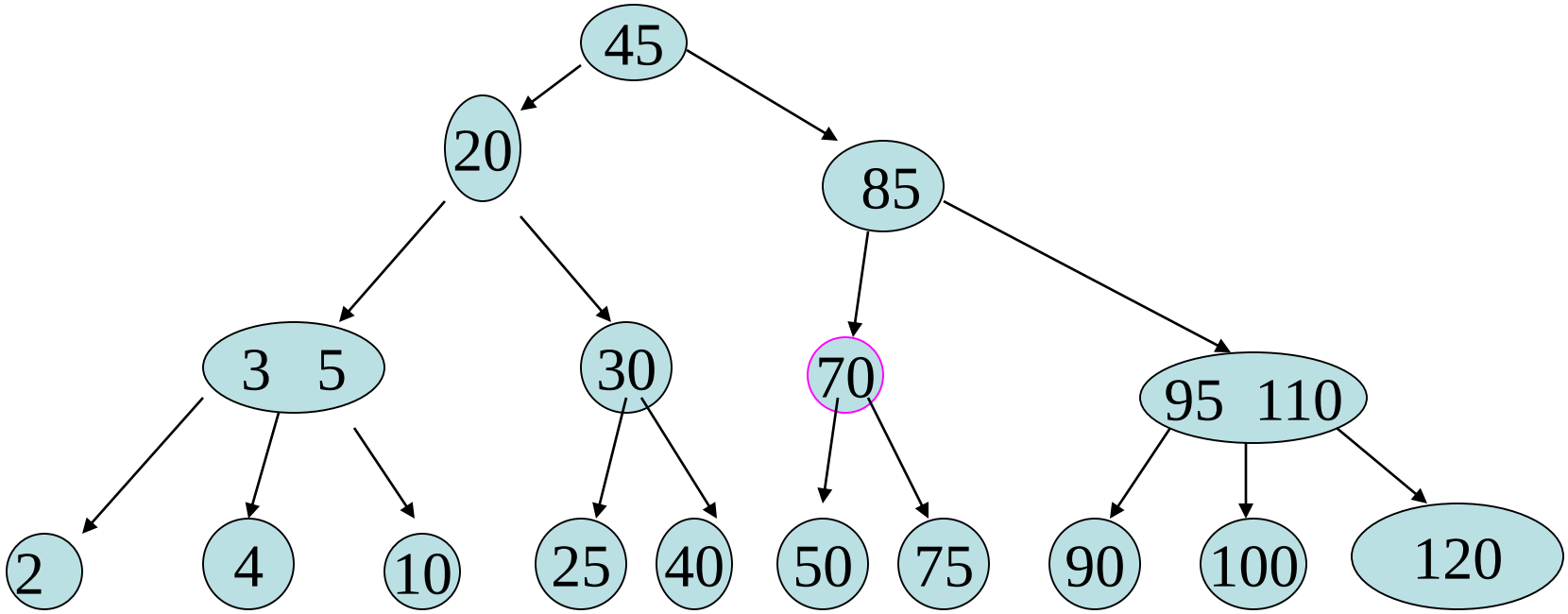


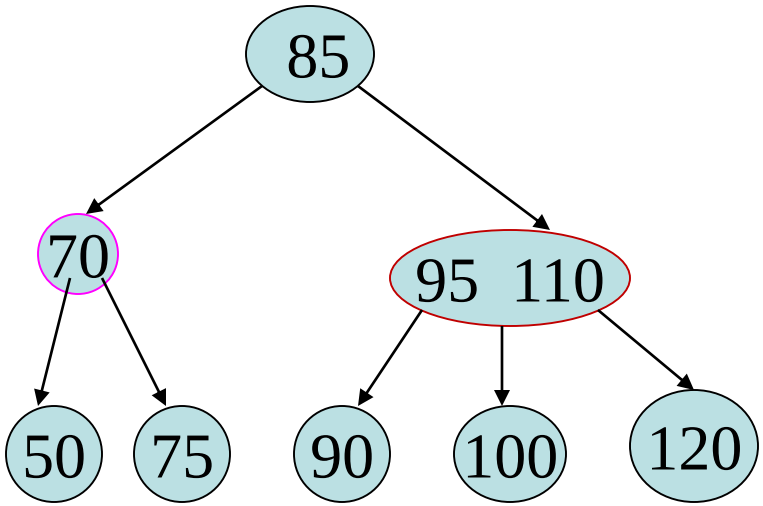
Re-distribute

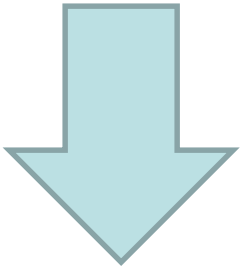
More on Deletion…

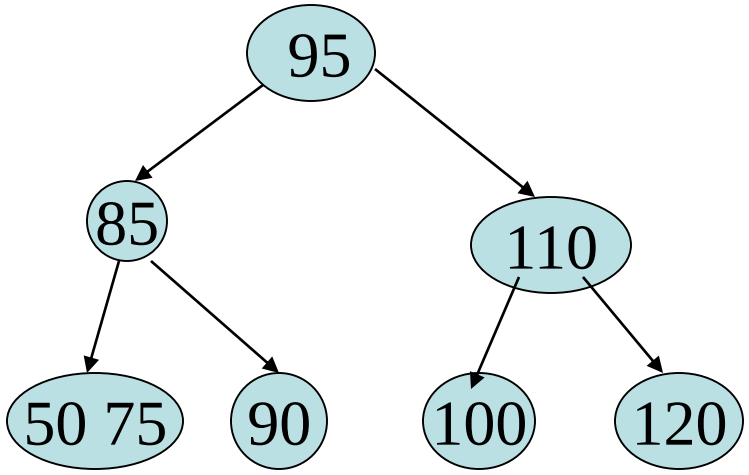
- If the item to be deleted is in a node with no other item and there are no adjacent 3-node or 4-node siblings, then we must bring down an item from the parent and place it in an adjacent sibling node
- We need to perform fusion operation, that is, merge nodes/subtrees
- Repeat recursively up the tree until no change is needed, or root is reached. (If root becomes empty, delete it)

Remove(70)



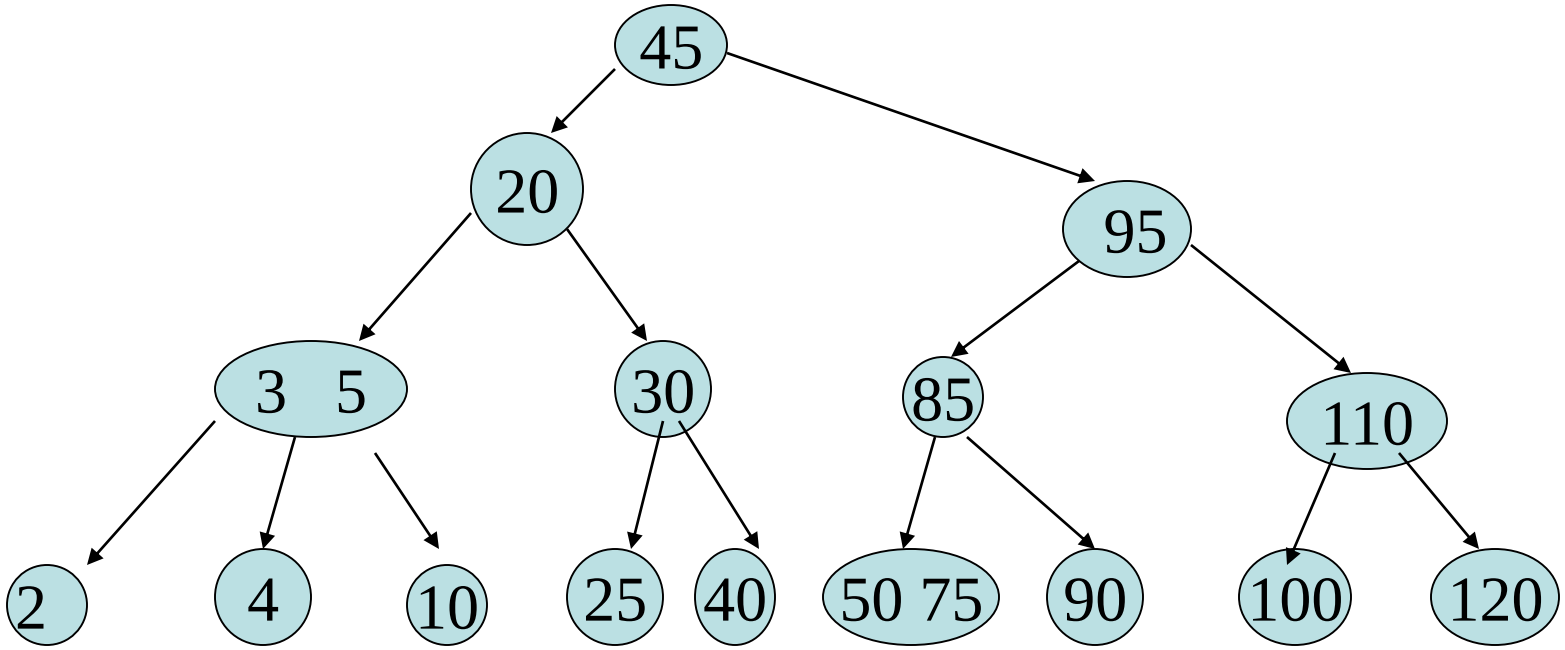






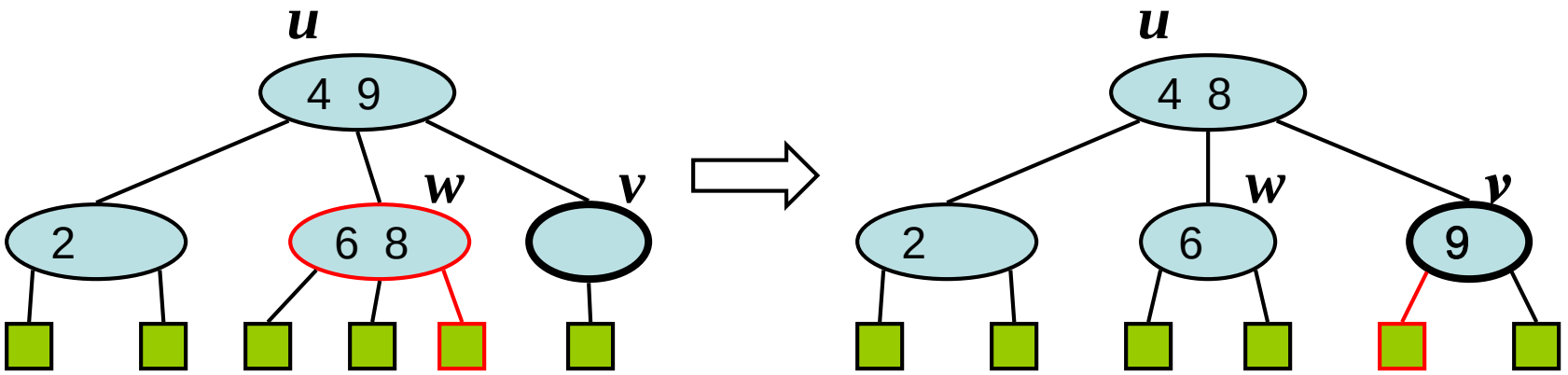
After redistributing the values

Resulting tree



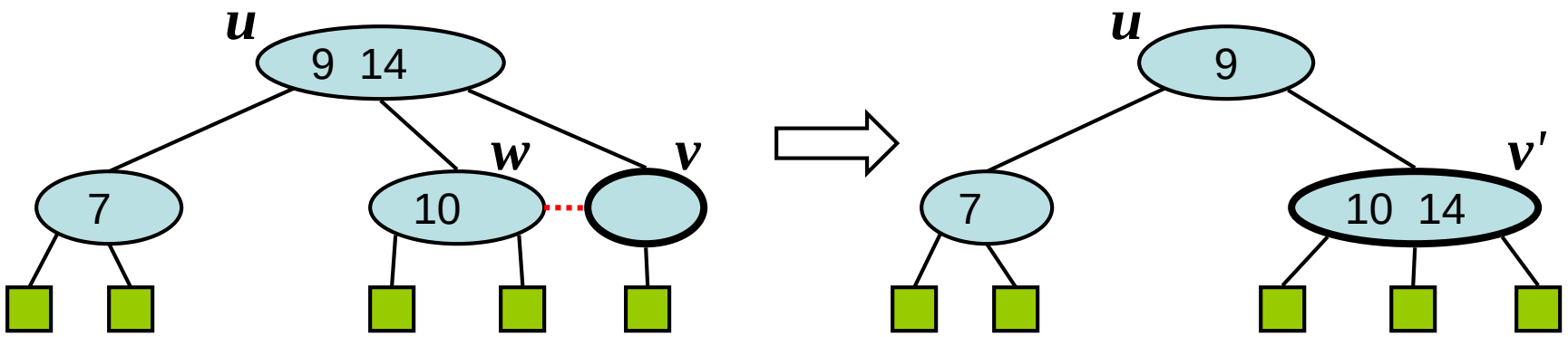
Underflow and Transfer

- Deleting an item from a node ***v*** may cause an underflow, where node ***v*** becomes a 1-node with one child and no keys
- To handle an underflow at node ***v*** with parent ***u***, we consider two cases
- **Case 1**: an adjacent sibling ***w*** of ***v*** is a 3-node or a 4-node
  - Transfer operation:
    1. move a child of ***w*** to ***v***
    2. move an item from ***u*** to ***v***
    3. move an item from ***w*** to ***u***
  - After a transfer, no underflow occurs



Underflow and Fusion

- **Case 2**: the adjacent siblings of ***v*** are 2-nodes
  - Fusion operation: merge ***v*** with an adjacent sibling ***w*** and move an item from ***u*** to the merged node ***v'***
  - After a merge, the underflow may propagate to the parent ***u***



Analysis of Deletion

- Let T be a (2,4) tree with n items
  - Tree T has $O(\log n)$ height
- For a deletion operation
  - visit $O(\log n)$ nodes to locate the node from which to delete the item
  - handle an underflow with a series of $O(\log n)$ merge operations, followed by at most one transfer
  - Each merge and transfer takes $O(1)$ time
- Therefore, deleting an item from a (2,4) tree takes $O(\log n)$ time